

[Django Object-Relational Mapping (ORM)] (CheatSheet)

1. Basic Queries

- **Retrieve All Records:** `ModelName.objects.all()`
- **Retrieve a Single Record:** `ModelName.objects.get(id=1)`
- **Filter Records:** `ModelName.objects.filter(attribute=value)`
- **Exclude Records:** `ModelName.objects.exclude(attribute=value)`
- **Order Records:** `ModelName.objects.order_by('attribute')`
- **Chain Filters:** `ModelName.objects.filter().exclude().order_by()`
- **Limit QuerySet:** `ModelName.objects.all()[:10]`

2. Field Lookups

- **Exact Match:** `ModelName.objects.filter(attribute__exact=value)`
- **Case-Insensitive Match:** `ModelName.objects.filter(attribute__iexact=value)`
- **Contains:** `ModelName.objects.filter(attribute__contains=value)`
- **Case-Insensitive Contains:**
`ModelName.objects.filter(attribute__icontains=value)`
- **Greater Than:** `ModelName.objects.filter(attribute__gt=value)`
- **Less Than:** `ModelName.objects.filter(attribute__lt=value)`
- **Greater Than or Equal To:** `ModelName.objects.filter(attribute__gte=value)`
- **Less Than or Equal To:** `ModelName.objects.filter(attribute__lte=value)`
- **In a List:** `ModelName.objects.filter(attribute__in=[value1, value2])`
- **Starts/Ends With:** `ModelName.objects.filter(attribute__startswith=value),`
`ModelName.objects.filter(attribute__endswith=value)`
- **Date Queries:** `ModelName.objects.filter(attribute__date=date(2020, 1, 1))`
- **Year, Month, Day Queries:** `ModelName.objects.filter(attribute__year=2020),`
similarly `__month` and `__day`
- **Range:** `ModelName.objects.filter(attribute__range=(start_value,`
`end_value))`

3. Querying Related Objects

- **Select Related for Foreign Key:**
`ModelName.objects.select_related('related_name').all()`
- **Prefetch Related for Many-to-Many or Reverse Foreign Key:**
`ModelName.objects.prefetch_related('related_name').all()`

- **Filter on Related Fields:**
`ModelName.objects.filter(related_name__field=value)`
- **Chaining Filters on Related Fields:**
`ModelName.objects.filter(related_name__field1=value1,
related_name__field2=value2)`
- **Related Fields Greater/Less Than:**
`ModelName.objects.filter(related_name__field__gt=value)`

4. Aggregation and Annotation

- **Sum:** `ModelName.objects.aggregate(Sum('field'))`
- **Average:** `ModelName.objects.aggregate(Avg('field'))`
- **Count:** `ModelName.objects.aggregate(Count('field'))`
- **Min/Max:** `ModelName.objects.aggregate(Min('field'), Max('field'))`
- **Annotate:** `ModelName.objects.annotate(field_count=Count('field'))`

5. Complex Queries with Q Objects

- **AND Conditions:** `ModelName.objects.filter(Q(field1=value1) &
Q(field2=value2))`
- **OR Conditions:** `ModelName.objects.filter(Q(field1=value1) |
Q(field2=value2))`
- **NOT Conditions:** `ModelName.objects.filter(~Q(field=value))`

6. Updating and Deleting

- **Update Records:**
`ModelName.objects.filter(attribute=value).update(field=new_value)`
- **Delete Records:** `ModelName.objects.filter(attribute=value).delete()`

7. Getting Data from QuerySets

- **Values (Dictionary) Representation:** `ModelName.objects.values('field1',
'field2')`
- **Values List:** `ModelName.objects.values_list('field1', flat=True)`
- **Iterating Over QuerySet:** `[item for item in ModelName.objects.all()]`

8. Handling Transactions

- **Atomic Transactions:** `with transaction.atomic(): # your operations`

9. Custom Manager Methods

- **Custom QuerySet Method in Manager:** `ModelName.objects.custom_method()`

10. Raw SQL Queries

- **Raw SQL Query:** `ModelName.objects.raw('SELECT * FROM app_modelname')`

11. Database Functions

- **Concatenate Fields:** `ModelName.objects.annotate(new_field=Concat('field1', Value(' '), 'field2'))`
- **Length Function:**
`ModelName.objects.annotate(text_length=Length('textfield'))`
- **Lower, Upper Functions:**
`ModelName.objects.annotate(lower_field=Lower('field'))`
- **Coalesce Function:**
`ModelName.objects.annotate(field_or_default=Coalesce('field', Value('default')))`
- **Cast Function:** `ModelName.objects.annotate(cast_field=Cast('char_field', IntegerField()))`

12. Pagination with QuerySets

- **Paginator:** `paginator = Paginator(ModelName.objects.all(), 10) # 10 items per page`

13. QuerySet Optimization

- **Only/Defer Fields:** `ModelName.objects.defer('field').only('other_field')`
- **Using Select Related:**
`ModelName.objects.select_related('foreign_key_field')`
- **Using Prefetch Related:**
`ModelName.objects.prefetch_related('many_to_many_field')`

14. QuerySet Existence Check

- **Existence of Records:** `ModelName.objects.filter(attribute=value).exists()`

15. Conditional Expressions

- **Case/When:** `ModelName.objects.annotate(new_field=Case(When(condition, then=Value('result')), default=Value('default')))`
- **Conditional Update:**
`ModelName.objects.filter(condition).update(field=Case(When(sub_condition, then=Value('value')), default=Value('default')))`

16. Database Schema Queries

- **Get SQL Schema of a Model:**
`str(ModelName.objects.model._meta.sql_schema())`

17. F Expressions for Dynamic Data Handling

- **Dynamic Field Operations:** `ModelName.objects.update(field=F('other_field') * 2)`

18. QuerySet None

- **Empty QuerySet:** `ModelName.objects.none()`

19. Database Locking

- **Select for Update:**
`ModelName.objects.select_for_update().filter(attribute=value)`

20. Distinct Queries

- **Distinct:** `ModelName.objects.distinct()`
- **Distinct on Fields:** `ModelName.objects.order_by('field').distinct('field')`

21. Extra Queries

- **Using Extra:** `ModelName.objects.extra(select={'is_recent': "pub_date > '2020-01-01'"})`

22. Combining QuerySets

- **Union of QuerySets:** `qs1.union(qs2)`
- **Intersection of QuerySets:** `qs1.intersection(qs2)`
- **Difference of QuerySets:** `qs1.difference(qs2)`

23. Specific Database Operations

- **Using Specific Database:** `ModelName.objects.using('database_name').all()`

24. Advanced Filtering

- **Chaining Complex Filters:** `ModelName.objects.filter().exclude().filter()`
- **Filtering with Q Objects and Aggregation:**
`ModelName.objects.filter(Q()).annotate()`

25. Model Instance Creation

- **Create and Save Instance:** `instance = ModelName(field=value);
instance.save()`
- **Create with Objects.create():** `instance =
ModelName.objects.create(field=value)`

26. Bulk Operations

- **Bulk Create:** `ModelName.objects.bulk_create([ModelName(field=value), ...])`
- **Bulk Update:** `ModelName.objects.bulk_update(objs, ['field'])`

27. Selecting Related Objects

- **Select Related for Performance:**
`ModelName.objects.select_related('related_name')`
- **Prefetch Related for Reverse Lookups:**
`ModelName.objects.prefetch_related('related_name_set')`

28. Advanced Date and Time Queries

- **Filtering by Date:** `ModelName.objects.filter(pub_date__date=date(2020, 1, 1))`
- **Filtering by Year, Month, Day:**
`ModelName.objects.filter(pub_date__year=2020, pub_date__month=1, pub_date__day=1)`

29. Slicing QuerySets

- **Slicing for Limit and Offset:** `ModelName.objects.all()[10:20]`

30. Iterating Over QuerySets

- **Iterate with Only Needed Fields:** `ModelName.objects.only('field1', 'field2').all()`

31. Checking for Empty QuerySets

- **Empty Check:** `if not ModelName.objects.exists(): # do something`

32. Miscellaneous Operations

- **Getting the Last Item:** `ModelName.objects.latest('pub_date')`
- **Getting the First Item:** `ModelName.objects.earliest('pub_date')`
- **Reversing the QuerySet:** `ModelName.objects.reverse()`
- **Count of Records:** `ModelName.objects.count()`
- **Describing Aggregate:** `ModelName.objects.aggregate(Count('field'), Max('field'), Min('field'))`

33. Advanced Usage of F Expressions

- **Updating with F Expressions:**
`ModelName.objects.update(field=F('other_field') + 1)`
- **Filtering with F Expressions:**
`ModelName.objects.filter(field__gt=F('other_field'))`

34. Working with Annotations

- **Annotation with Aggregate Functions:**
`ModelName.objects.annotate(total=Sum('field'))`
- **Conditional Annotation:**
`ModelName.objects.annotate(is_new=Case(When(condition, then=Value(True)), default=Value(False)))`

35. Working with GeoDjango

- **GeoDjango Distance Filter:**
`ModelName.objects.filter(location__distance_lte=(point, D(km=5)))`
- **GeoDjango Area Calculation:**
`ModelName.objects.annotate(area=Area('geometry'))`

36. Model Inheritance Handling

- **Querying Child Models:**

```
ParentModel.objects.select_related('childmodel').all()
```

37. Database Index Related Queries

- **Forcing a Specific Index:**

```
ModelName.objects.force_index('index_name').filter()
```

38. Query Hints for Optimizations

- **Using Query Hints:** `ModelName.objects.using_hint('hint_name').filter()`

39. Complex Joins and Subqueries

- **Subquery in QuerySet:** `subquery =`

```
Subquery(AnotherModel.objects.filter(relation=F('outer_ref_field')).values('field')[:1])
```

40. Working with JSON Fields

- **Filtering on JSON Field:** `ModelName.objects.filter(json_field__key=value)`
- **Querying Inside a JSON Field Array:**

```
ModelName.objects.filter(json_field__contains=[{'key': 'value'}])
```

41. Raw Queries and Expressions

- **Using Raw SQL Queries:** `ModelName.objects.raw('SELECT * FROM myapp_modelname WHERE condition')`
- **Raw SQL for Updates:** `ModelName.objects.raw('UPDATE myapp_modelname SET field=value WHERE condition')`

42. Caching QuerySets

- **Caching Results of QuerySets:** `from django.core.cache import cache; cache.set('my_key', ModelName.objects.all(), 300)`

43. Query Optimization Techniques

- **Select Only for Performance:** `ModelName.objects.only('field1', 'field2')`

- **Defer Fields Not Needed Immediately:**
`ModelName.objects.defer('large_field')`

44. Advanced **Date/Time** Handling

- **Truncating Dates:** `ModelName.objects.annotate(date=Trunc('pub_date', 'day', output_field=DateField()))`